

Centro de Procesamiento de Datos



UNIVERSIDAD
DE GRANADA

Práctica 2. Contenedores Docker (II)

Juan Navarro Maldonado

Índice:

Creación de una imagen personalizada de Docker con Dockerfile y un index.html personalizado.....	3
Subir imagen a hub.docker.com.....	3
Contenido del Dockerfile y ficheros utilizados.....	3
Iniciar servidor Wordpress y editar la página principal.....	3
Opcional: Preparar imagen personalizada para realizar test de evaluacion de rendimiento y subir a hub.docker.com.....	3

Creación de una imagen personalizada de Docker con Dockerfile y un index.html personalizado

Esta parte la realizaremos con los códigos que nos presenta en el apartado 1 del guión de prácticas, donde utilizaremos el dockerfile básico para la creación de una imagen Debian con apache2 instalado junto con un index.html básico que nos sirva a modo de comprobación que se ha instalado todo correctamente.

```
FROM debian
MAINTAINER Usuario CPD "juannavarrom@correo.ugr.es"
RUN apt-get update && apt-get install -y apache2 && apt-get clean
&& rm -rf /var/lib/apt/lists/*
ENV APACHE_RUN_USER www-data
ENV APACHE_RUN_GROUP www-data
ENV APACHE_LOG_DIR /var/log/apache2
EXPOSE 80
ADD ["index.html", "/var/www/html/"]
ENTRYPOINT ["/usr/sbin/apache2ctl", "-D", "FOREGROUND"]
```

Código 1: Dockerfile ejemplo

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Prueba CPD</title>
  </head>
  <body>
    <h1>Prueba inicial CPD</h1>
  </body>
</html>
```

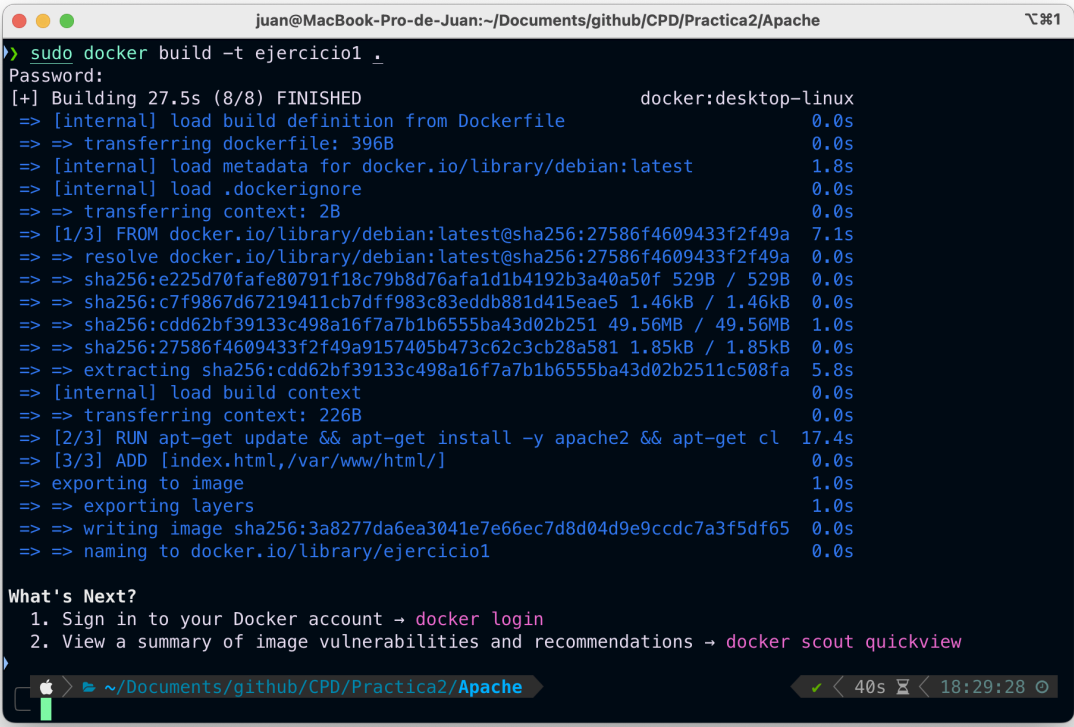
Código 2: Index.html ejemplo

Una vez tengamos nuestros ficheros creados en un directorio de nuestra máquina local, procederemos a crear la imagen de Docker para poder subirla a hub.docker.com. Podemos crear nuestra nueva imagen con el siguiente comando:

```
sudo docker build -t ejercicio1 .
```

Comando 1: Creación imagen personalizada

Si la imagen se ha creado correctamente nos saldrá el siguiente mensaje por pantalla:



```
juan@MacBook-Pro-de-Juan:~/Documents/github/CPD/Practica2/Apache
> sudo docker build -t ejercicio1 .
Password:
[+] Building 27.5s (8/8) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile               0.0s
=> => transferring dockerfile: 396B                               0.0s
=> [internal] load metadata for docker.io/library/debian:latest  1.8s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 2B                                     0.0s
=> [1/3] FROM docker.io/library/debian:latest@sha256:27586f4609433f2f49a 7.1s
=> => resolve docker.io/library/debian:latest@sha256:27586f4609433f2f49a 0.0s
=> => sha256:e225d70fafa80791f18c79b8d76afa1d1b4192b3a40a50f 529B / 529B 0.0s
=> => sha256:c7f9867d67219411cb7dff983c83eddb881d415eae5 1.46kB / 1.46kB 0.0s
=> => sha256:cdd62bf39133c498a16f7a7b1b6555ba43d02b251 49.56MB / 49.56MB 1.0s
=> => sha256:27586f4609433f2f49a9157405b473c62c3cb28a581 1.85kB / 1.85kB 0.0s
=> => extracting sha256:cdd62bf39133c498a16f7a7b1b6555ba43d02b251c508fa 5.8s
=> [internal] load build context                                  0.0s
=> => transferring context: 226B                                    0.0s
=> [2/3] RUN apt-get update && apt-get install -y apache2 && apt-get cl 17.4s
=> [3/3] ADD [index.html,/var/www/html/]                        0.0s
=> exporting to image                                           1.0s
=> => exporting layers                                             1.0s
=> => writing image sha256:3a8277da6ea3041e7e66ec7d8d04d9e9ccdc7a3f5df65 0.0s
=> => naming to docker.io/library/ejercicio1                      0.0s

What's Next?
 1. Sign in to your Docker account → docker login
 2. View a summary of image vulnerabilities and recommendations → docker scout quickview

~ /Documents/github/CPD/Practica2/Apache 40s 18:29:28
```

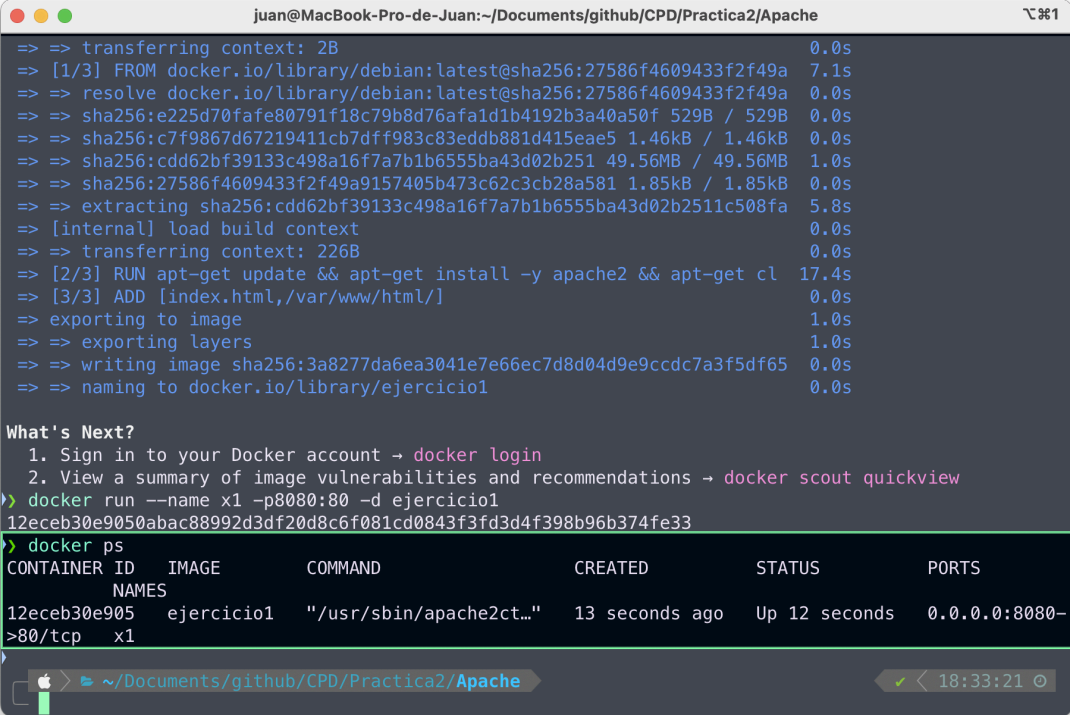
Imagen 1: Creación imagen personalizada

El siguiente paso es levantar el contenedor para que podamos visualizar nuestro index.html que hemos creado, para ello ejecutaremos el siguiente comando:

```
docker run --name x1 -p8080:80 -d ejercicio1
```

Comando 2: Levantar el contenedor creado

Con esto podremos hacer un docker ps para ver si se ha creado correctamente y no esta dando fallos:



```
=> => transferring context: 2B 0.0s
=> [1/3] FROM docker.io/library/debian:latest@sha256:27586f4609433f2f49a 7.1s
=> => resolve docker.io/library/debian:latest@sha256:27586f4609433f2f49a 0.0s
=> => sha256:e225d70fafe80791f18c79b8d76afa1d1b4192b3a40a50f 529B / 529B 0.0s
=> => sha256:c7f9867d67219411cb7dff983c83eddb881d415eae5 1.46kB / 1.46kB 0.0s
=> => sha256:cdd62bf39133c498a16f7a7b1b6555ba43d02b251 49.56MB / 49.56MB 1.0s
=> => sha256:27586f4609433f2f49a9157405b473c62c3cb28a581 1.85kB / 1.85kB 0.0s
=> => extracting sha256:cdd62bf39133c498a16f7a7b1b6555ba43d02b251c508fa 5.8s
=> [internal] load build context 0.0s
=> => transferring context: 226B 0.0s
=> [2/3] RUN apt-get update && apt-get install -y apache2 && apt-get cl 17.4s
=> [3/3] ADD [index.html,/var/www/html/] 0.0s
=> exporting to image 1.0s
=> => exporting layers 1.0s
=> => writing image sha256:3a8277da6ea3041e7e66ec7d8d04d9e9ccdc7a3f5df65 0.0s
=> => naming to docker.io/library/ejercicio1 0.0s

What's Next?
1. Sign in to your Docker account → docker login
2. View a summary of image vulnerabilities and recommendations → docker scout quickview
> docker run --name x1 -p8080:80 -d ejercicio1
12eceb30e9050abac88992d3df20d8c6f081cd0843f3fd3d4f398b96b374fe33
> docker ps
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
12eceb30e905	ejercicio1	"/usr/sbin/apache2ct..."	13 seconds ago	Up 12 seconds	0.0.0.0:8080->80/tcp

```
x1
```

Imagen 2: Verificación del contenedor

El último paso es verificar el index.html para ver que está todo correctamente configurado y listo para subir a hub.docker, para ello accedemos a la dirección localhost:8080 que hemos especificado con anterioridad al levantar el contenedor.

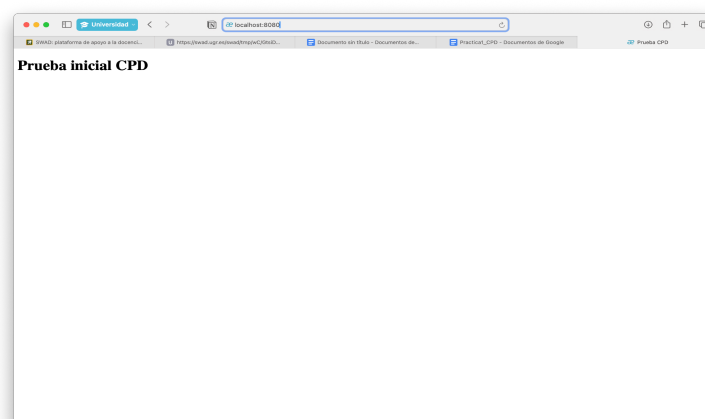


Imagen 3: Verificación del Index

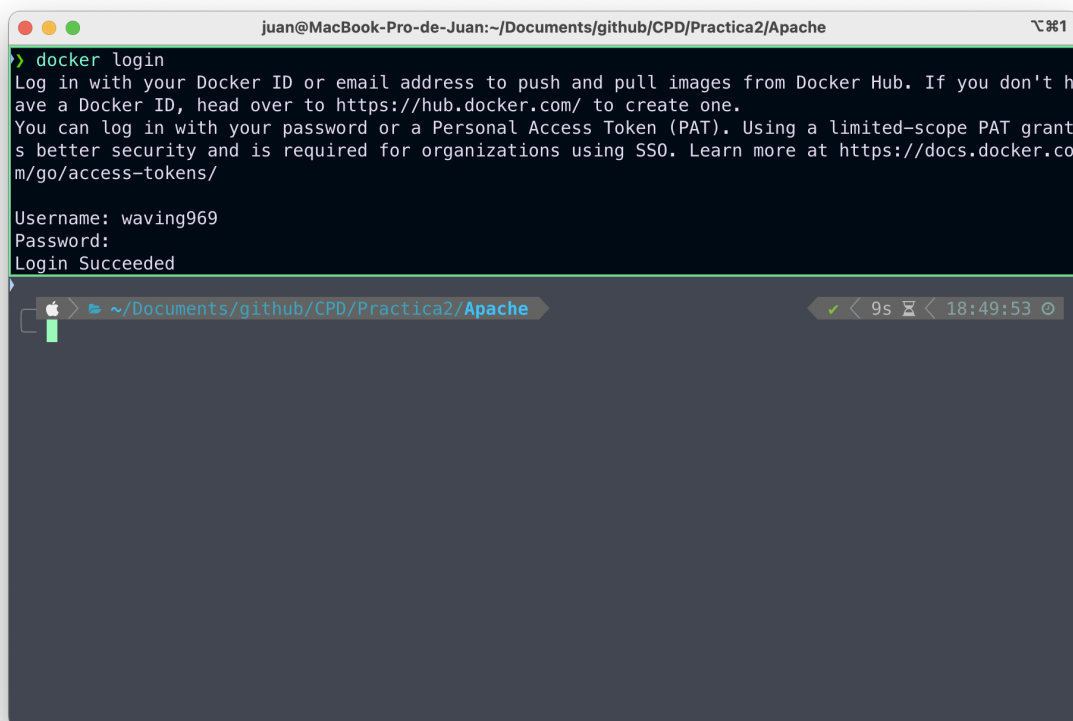
Subir imagen a hub.docker.com

Lo primero que tenemos que hacer es iniciar sesión con nuestra cuenta de docker, en el caso de que no tengamos cuenta, tendremos que crearnos una, yo voy a introducir las credenciales de mi cuenta para esta práctica, para iniciar sesión introduciremos el siguiente comando:

```
docker login
```

Comando 3: Inicio de sesión en hub.docker.com

Ahora nos pedirá las credenciales de nuestra cuenta, las ingresamos y ya habíamos iniciado sesión en nuestra máquina local:



```
juan@MacBook-Pro-de-Juan:~/Documents/github/CPD/Practica2/Apache
> docker login
Log in with your Docker ID or email address to push and pull images from Docker Hub. If you don't have a Docker ID, head over to https://hub.docker.com/ to create one.
You can log in with your password or a Personal Access Token (PAT). Using a limited-scope PAT grant is better security and is required for organizations using SSO. Learn more at https://docs.docker.com/go/access-tokens/

Username: waving969
Password:
Login Succeeded
```

Imagen 4: Inicio de sesión en hub.docker.com

Ahora tendremos que construir una nueva imagen con el nombre que queremos subir a nuestro repositorio de docker, en mi caso la voy a llamar como en el guion de la práctica waving969/cpd:1.0 y posteriormente la subiré al repositorio mediante un push:

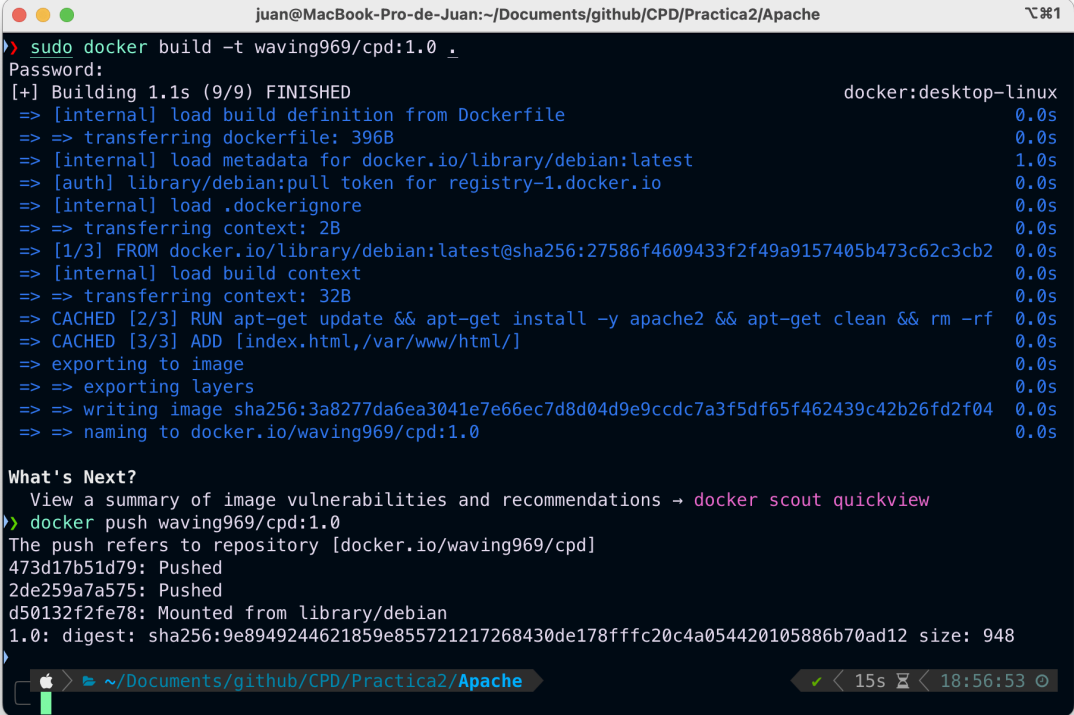
```
sudo docker build -t waving969/cpd:1.0 .
```

Comando 4: Creación de la nueva imagen

```
docker push waving969/cpd:1.0
```

Comando 5: Subir imagen a nuestro repositorio

De esta forma si lo hemos realizado correctamente nos deberían de salir los siguientes mensajes:



```
juan@MacBook-Pro-de-Juan:~/Documents/github/CPD/Practica2/Apache
> sudo docker build -t waving969/cpd:1.0 .
Password:
[+] Building 1.1s (9/9) FINISHED                                docker:desktop-linux
=> [internal] load build definition from Dockerfile              0.0s
=> => transferring dockerfile: 396B                             0.0s
=> [internal] load metadata for docker.io/library/debian:latest 1.0s
=> [auth] library/debian:pull token for registry-1.docker.io   0.0s
=> [internal] load .dockerignore                                0.0s
=> => transferring context: 2B                                    0.0s
=> [1/3] FROM docker.io/library/debian:latest@sha256:27586f4609433f2f49a9157405b473c62c3cb2 0.0s
=> [internal] load build context                                0.0s
=> => transferring context: 32B                                   0.0s
=> CACHED [2/3] RUN apt-get update && apt-get install -y apache2 && apt-get clean && rm -rf 0.0s
=> CACHED [3/3] ADD [index.html,/var/www/html/]                 0.0s
=> exporting to image                                           0.0s
=> => exporting layers                                           0.0s
=> => writing image sha256:3a8277da6ea3041e7e66ec7d8d04d9e9ccdc7a3f5df65f462439c42b26fd2f04 0.0s
=> => naming to docker.io/waving969/cpd:1.0                     0.0s

What's Next?
View a summary of image vulnerabilities and recommendations -> docker scout quickview
> docker push waving969/cpd:1.0
The push refers to repository [docker.io/waving969/cpd]
473d17b51d79: Pushed
2de259a7a575: Pushed
d50132f2fe78: Mounted from library/debian
1.0: digest: sha256:9e8949244621859e855721217268430de178fffc20c4a054420105886b70ad12 size: 948
```

Imagen 5: Creación de la imagen y subida al repositorio

Alternativamente podemos verificarlo a través de la página web de hub.docker en nuestros repositorios:

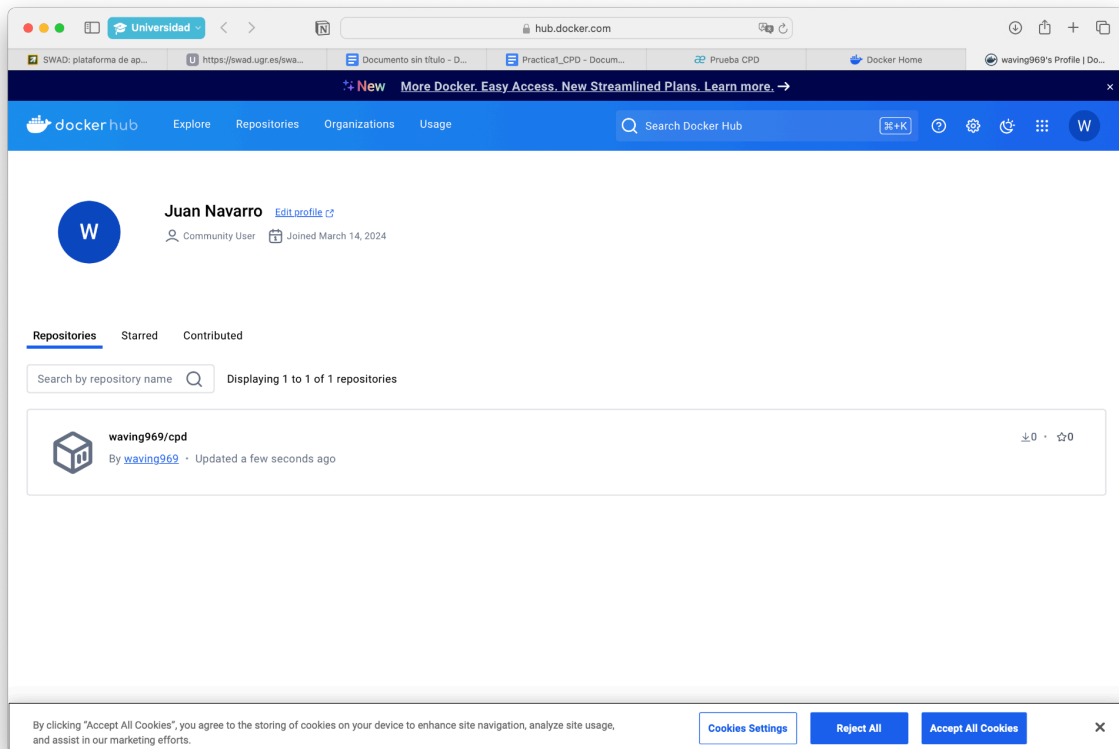


Imagen 6: Verificación en el repositorio

Contenido del Dockerfile y ficheros utilizados

Los ficheros utilizados para subir la imagen son los que he mencionado en el [apartado 1 de este documento](#)

Iniciar servidor Wordpress y editar la página principal

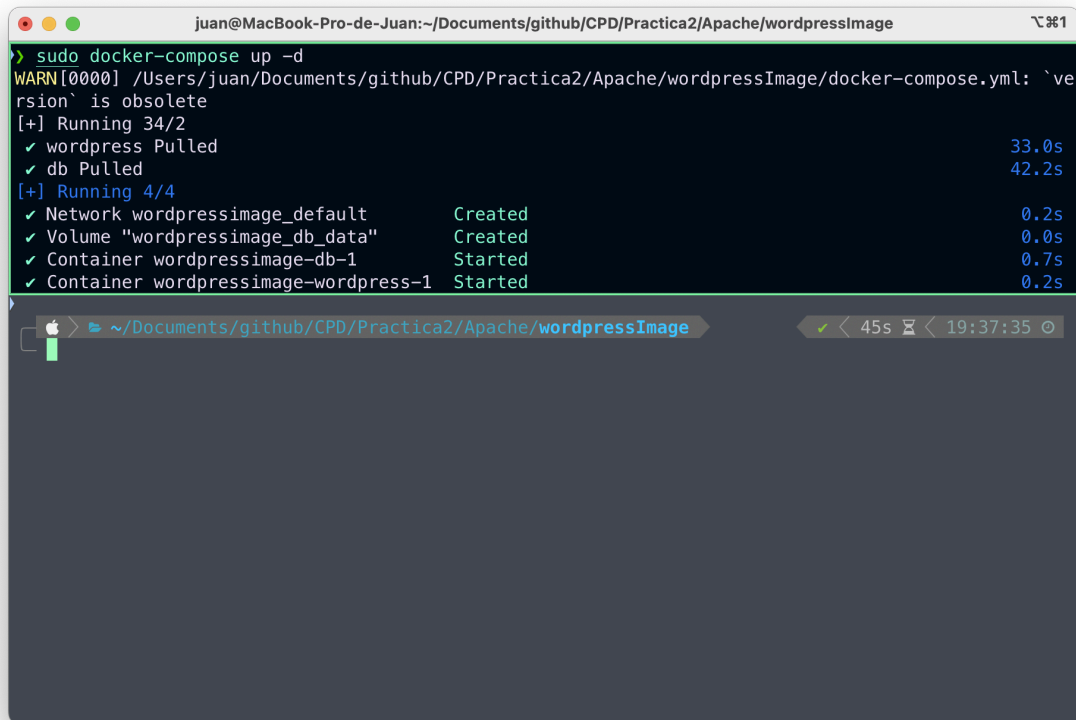
Lo primero será crear un docker-compose personalizado para wordpress, el cual nos creará dos imágenes para nuestro entorno de pruebas.

```
version: '3.3'
services:
  db:
    image: mysql:5.7
    volumes:
      - db_data:/var/lib/mysql
    restart: always
    environment:
      MYSQL_ROOT_PASSWORD: somewordpress
      MYSQL_DATABASE: wordpress
      MYSQL_USER: wordpress
      MYSQL_PASSWORD: cpdwordpress

  wordpress:
    depends_on:
      - db
    image: wordpress:latest
    ports:
      - "8000:80"
    restart: always
    environment:
      WORDPRESS_DB_HOST: db:3306
      WORDPRESS_DB_USER: wordpress
      WORDPRESS_DB_PASSWORD: cpdwordpress
volumes:
  db_data: {}
```

Código 5: docker-compose.yml

Una vez tengamos la imagen creada y montada veremos la siguiente informacion por pantalla:

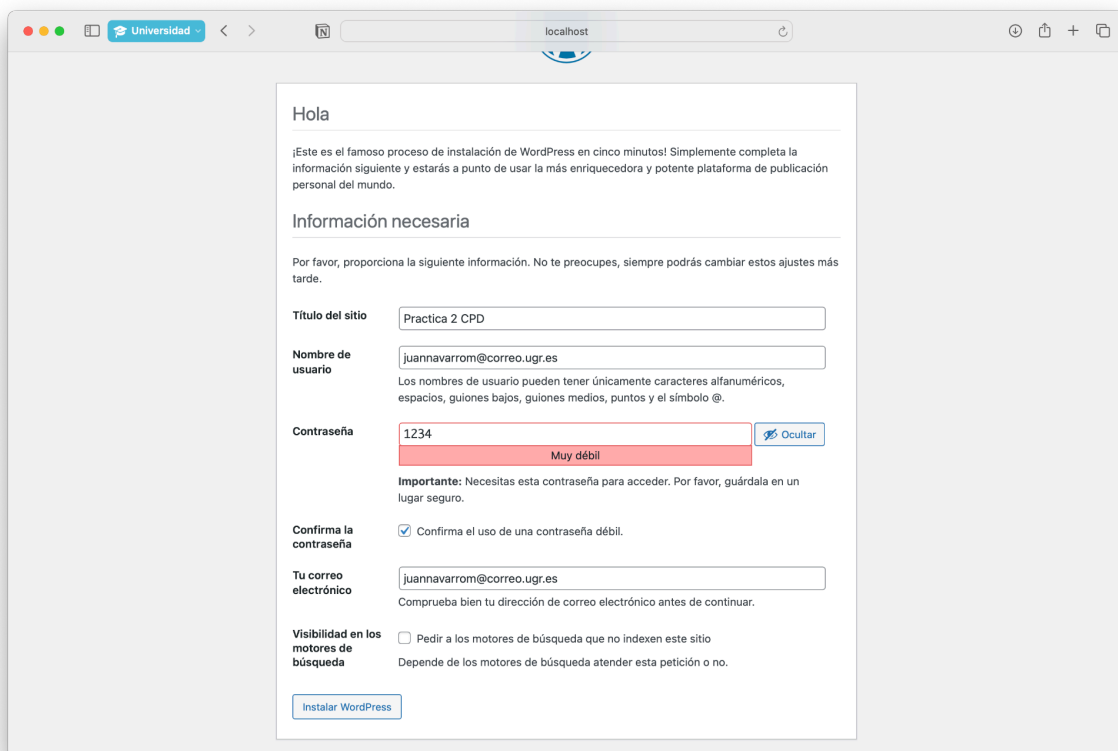


```
juan@MacBook-Pro-de-Juan: ~/Documents/github/CPD/Practica2/Apache/wordpressImage
> sudo docker-compose up -d
WARN[0000] /Users/juan/Documents/github/CPD/Practica2/Apache/wordpressImage/docker-compose.yml: `version` is obsolete
[+] Running 3/2
  ✓ wordpress Pulled                                33.0s
  ✓ db Pulled                                         42.2s
[+] Running 4/4
  ✓ Network wordpressimage_default                  Created      0.2s
  ✓ Volume "wordpressimage_db_data"                 Created      0.0s
  ✓ Container wordpressimage-db-1                   Started       0.7s
  ✓ Container wordpressimage-wordpress-1            Started       0.2s
```

The screenshot shows a macOS terminal window with the title 'juan@MacBook-Pro-de-Juan: ~/Documents/github/CPD/Practica2/Apache/wordpressImage'. The user has executed the command 'sudo docker-compose up -d'. The output shows a warning about the 'version' field in the docker-compose.yml file being obsolete. It then shows the progress of pulling the 'wordpress' and 'db' images, followed by creating the 'wordpressimage_default' network, the 'wordpressimage_db_data' volume, and starting the 'wordpressimage-db-1' and 'wordpressimage-wordpress-1' containers. A status bar at the bottom indicates a success checkmark, a duration of 45s, and a timestamp of 19:37:35.

Imagen 7: Creación del contenedor wordpress

Podemos verificarlo entrando a localhost:8000:



The screenshot shows the WordPress installation wizard in a web browser. The page has a light gray background with a white content area. At the top, it says "Hola" and provides a brief introduction to WordPress. Below this, there's a section titled "Información necesaria" (Necessary information). It asks for the site title, username, password, and email. The site title is "Practica 2 CPD". The username is "juannavarrom@correo.ugr.es". The password is "1234", which is marked as "Muy débil" (Very weak). There's a checkbox for "Confirma la contraseña" (Confirm password) which is checked. The email is "juannavarrom@correo.ugr.es". At the bottom, there's a checkbox for "Visibilidad en los motores de búsqueda" (Visibility in search engines) which is unchecked. A blue button labeled "Instalar WordPress" is at the bottom.

Hola

¡Este es el famoso proceso de instalación de WordPress en cinco minutos! Simplemente completa la información siguiente y estarás a punto de usar la más enriquecedora y potente plataforma de publicación personal del mundo.

Información necesaria

Por favor, proporciona la siguiente información. No te preocupes, siempre podrás cambiar estos ajustes más tarde.

Título del sitio

Nombre de usuario
Los nombres de usuario pueden tener únicamente caracteres alfanuméricos, espacios, guiones bajos, guiones medios, puntos y el símbolo @.

Contraseña [Ocultar](#)
Muy débil

Importante: Necesitas esta contraseña para acceder. Por favor, guárdala en un lugar seguro.

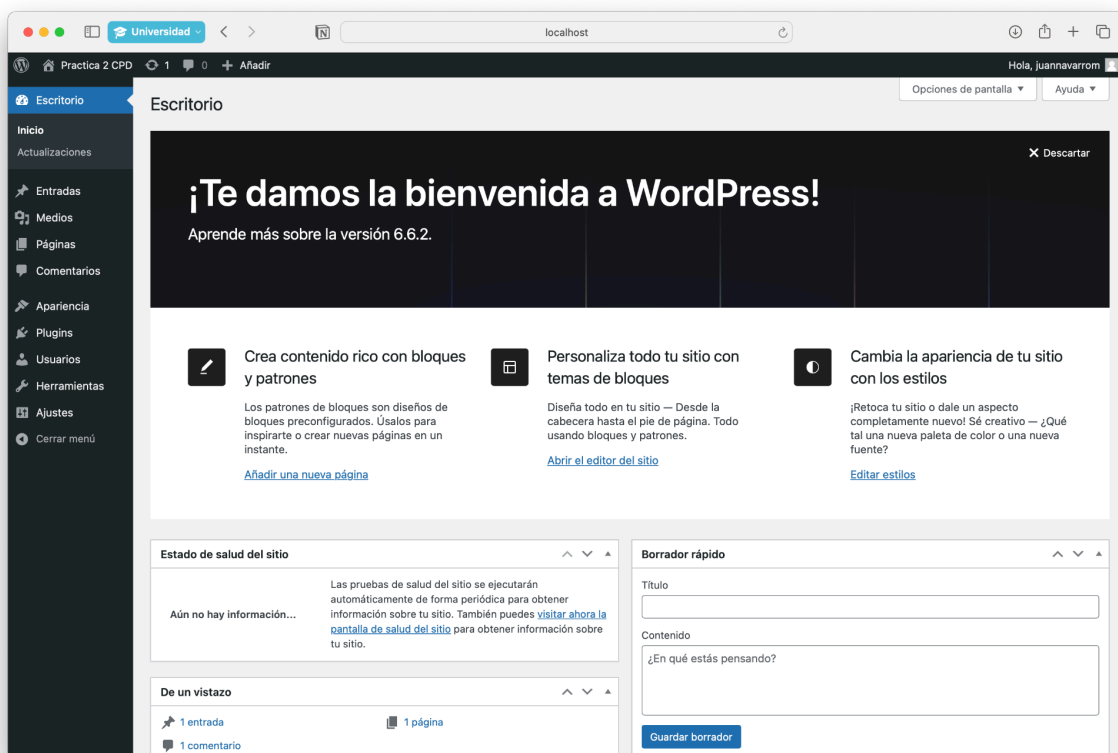
Confirma la contraseña ☒ Confirma el uso de una contraseña débil.

Tu correo electrónico
Comprueba bien tu dirección de correo electrónico antes de continuar.

Visibilidad en los motores de búsqueda ☐ Pedir a los motores de búsqueda que no indexen este sitio
Depende de los motores de búsqueda atender esta petición o no.

[Instalar WordPress](#)

Imagen 8: Configuración de nuestra pagina wordpress



The screenshot shows the WordPress dashboard. The left sidebar has a menu with "Inicio" (Home) selected. The main content area has a large banner that says "¡Te damos la bienvenida a WordPress!" (We welcome you to WordPress!) and "Aprende más sobre la versión 6.6.2." (Learn more about version 6.6.2.). Below the banner are three cards: "Crea contenido rico con bloques y patrones" (Create rich content with blocks and patterns), "Personaliza todo tu sitio con temas de bloques" (Customize your entire site with block themes), and "Cambia la apariencia de tu sitio con los estilos" (Change the look of your site with styles). At the bottom, there are three widgets: "Estado de salud del sitio" (Site health status), "De un vistazo" (At a glance), and "Borrador rápido" (Quick draft).

Hola, juannavarrom

Opciones de pantalla Ayuda

¡Te damos la bienvenida a WordPress!

Aprende más sobre la versión 6.6.2.

Crea contenido rico con bloques y patrones
Los patrones de bloques son diseños de bloques preconfigurados. Úsalos para inspirarte o crear nuevas páginas en un instante.
[Añadir una nueva página](#)

Personaliza todo tu sitio con temas de bloques
Diseña todo en tu sitio — Desde la cabecera hasta el pie de página. Todo usando bloques y patrones.
[Abrir el editor del sitio](#)

Cambia la apariencia de tu sitio con los estilos
¡Retoca tu sitio o dale un aspecto completamente nuevo! Sé creativo — ¿Qué tal una nueva paleta de color o una nueva fuente?
[Editar estilos](#)

Estado de salud del sitio
Aún no hay información... Las pruebas de salud del sitio se ejecutarán automáticamente de forma periódica para obtener información sobre tu sitio. También puedes [visitar ahora la pantalla de salud del sitio](#) para obtener información sobre tu sitio.

De un vistazo
1 entrada 1 página 1 comentario

Borrador rápido
Título
Contenido
¿En qué estás pensando?
[Guardar borrador](#)

Imagen 9: Página administración de wordpress

Y por ultimo editamos una página para poder mostrarla por pantalla:

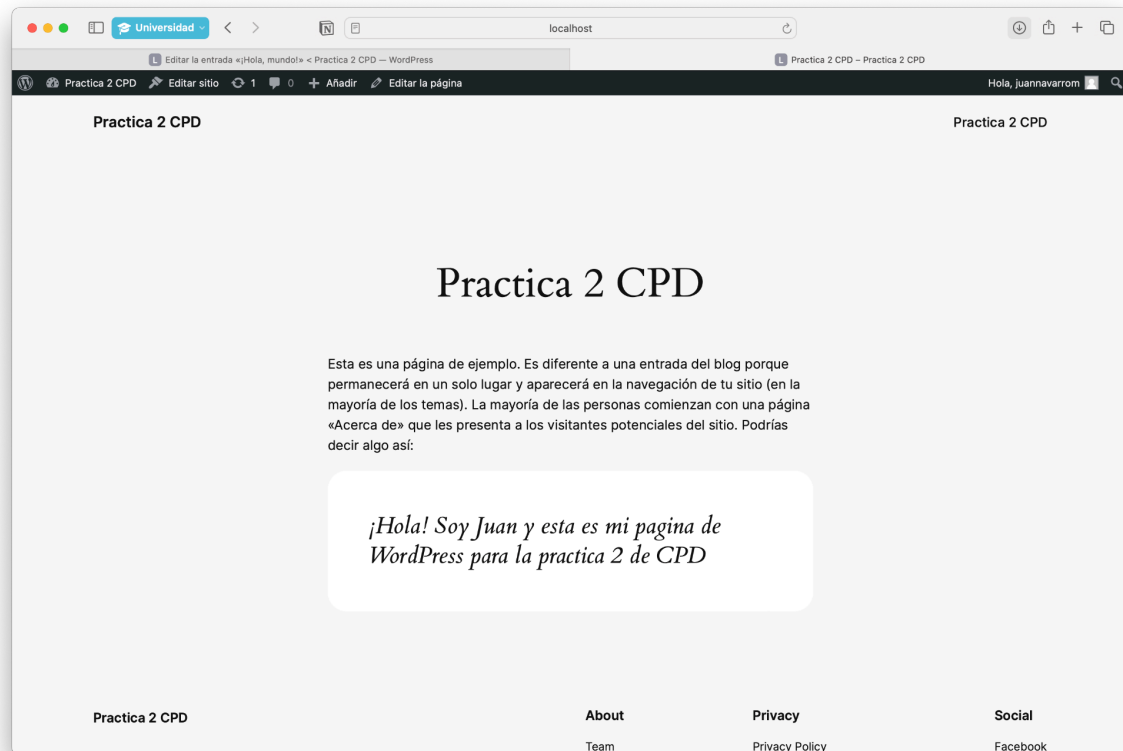


Imagen 10: Página de ejemplo para probar nuestro servidor wordpress

Opcional: Preparar imagen personalizada para realizar test de evaluacion de rendimiento y subir a hub.docker.com

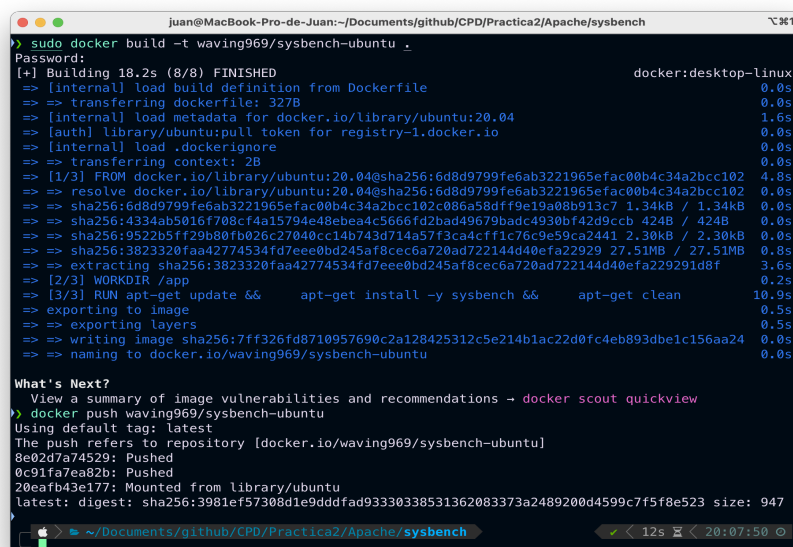
Para este apartado vamos a proceder a usar la imagen de ubuntu por defecto con la herramienta sysbench instalada, para ello vamos a crear un dockerfile personalizado como el siguiente:

```
# Usar la imagen base de Ubuntu
FROM ubuntu:20.04
# Establecer el directorio de trabajo
WORKDIR /app
# Actualizar e instalar sysbench
RUN apt-get update && \
    apt-get install -y sysbench && \
    apt-get clean

# Comando por defecto para ejecutar sysbench
CMD ["sysbench", "--help"]
```

Código 6: Dockerfile sysbench-ubuntu

Creamos la imagen como hemos visto en apartado 1 de esta práctica para exportarla a nuestro repositorio de hub.docker.com:

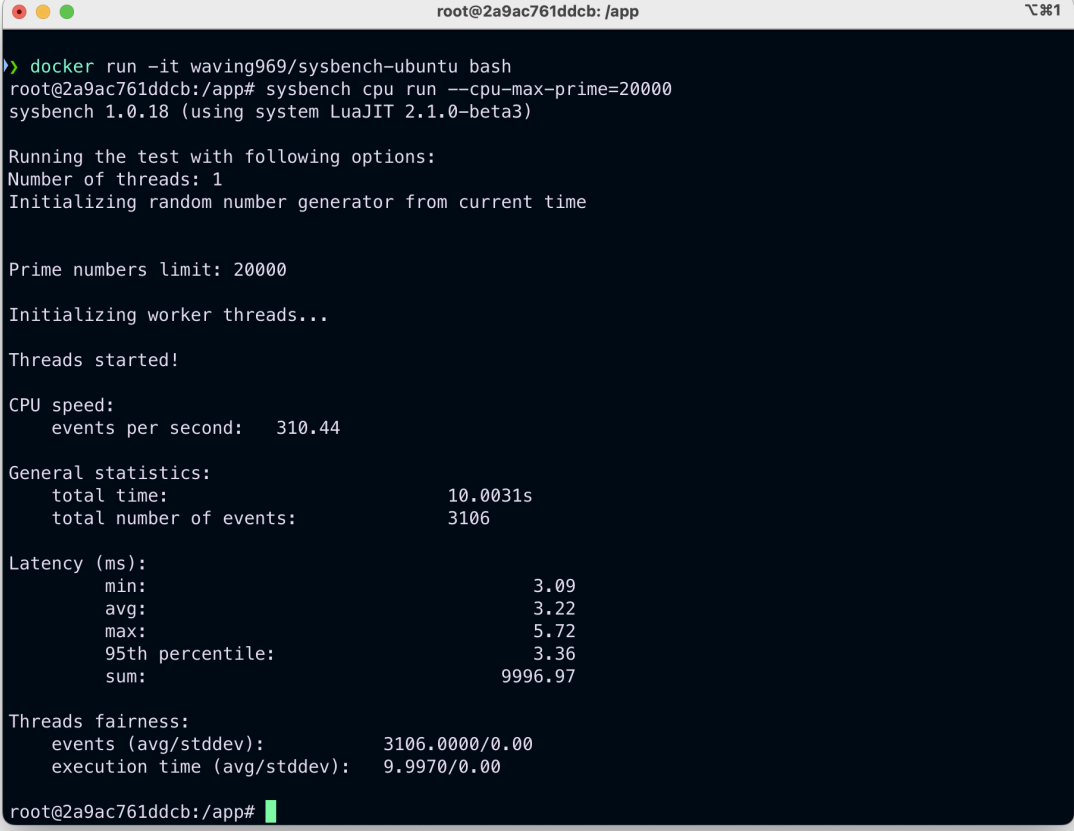


```
juan@MacBook-Pro-de-Juan: ~/Documents/github/CPD/Practica2/Apache/sysbench
>> sudo docker build -t waving969/sysbench-ubuntu .
Password:
[+] Building 18.2s (8/8) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 327B
=> [internal] load metadata for docker.io/library/ubuntu:20.04
=> [auth] library/ubuntu:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/3] FROM docker.io/library/ubuntu:20.04@sha256:6d8d9799fe6ab3221965efac00b4c34a2bcc102
=> => resolve docker.io/library/ubuntu:20.04@sha256:6d8d9799fe6ab3221965efac00b4c34a2bcc102
=> sha256:6d8d9799fe6ab3221965efac00b4c34a2bcc102c086a58dff9e19a08b913c7 1.34kB / 1.34kB
=> sha256:4334ab5016f708cf4a15794e48ebee4c5666fd2bad49679badc4930bf42d9ccb 424B / 424B
=> sha256:9522b5ff29b80fb026c27040cc14b743d714a57f3ca4cfff1c76c9e59ca2441 2.30kB / 2.30kB
=> sha256:3823320faa42774534fd7eee0bd245af8cec6a720ad722144d40efa22929 27.51MB / 27.51MB
=> => extracting sha256:3823320faa42774534fd7eee0bd245af8cec6a720ad722144d40efa229291d8f 3.6s
=> [2/3] WORKDIR /app
=> [3/3] RUN apt-get update && apt-get install -y sysbench && apt-get clean
=> exporting to image
=> exporting layers
=> writing image sha256:7ff326fd8710957690c2a128425312c5e214b1ac22d0fc4eb893dbe1c156aa24
=> naming to docker.io/waving969/sysbench-ubuntu

What's Next?
View a summary of image vulnerabilities and recommendations -> docker scout quickview
>> docker push waving969/sysbench-ubuntu
Using default tag: latest
The push refers to repository [docker.io/waving969/sysbench-ubuntu]
8e02d7a74529: Pushed
0c91fa7ea82b: Pushed
20eafb43e177: Mounted from library/ubuntu
latest: digest: sha256:3981ef57308d1e9dddfad93330338531362083373a2489200d4599c7f5f8e523 size: 947
```

Imagen 11: Creación de la imagen y push al repositorio

Una vez tengamos la imagen creada procederemos a realizar las pruebas con sysbench, para tener una referencia, primero ejecutaremos el benchmark sin parámetros y posteriormente limitaremos las CPUs para ver los resultados:

A terminal window with a dark blue background and white text. The window title is 'root@2a9ac761ddcb: /app'. The terminal shows the execution of 'sysbench cpu run --cpu-max-prime=20000'. It displays various statistics including CPU speed (310.44 events per second), general statistics (total time 10.0031s, total events 3106), latency (avg 3.22ms), and threads fairness (events 3106.0000/0.00, execution time 9.9970/0.00).

```
root@2a9ac761ddcb: /app
> docker run -it waving969/sysbench-ubuntu bash
root@2a9ac761ddcb:/app# sysbench cpu run --cpu-max-prime=20000
sysbench 1.0.18 (using system LuaJIT 2.1.0-beta3)

Running the test with following options:
Number of threads: 1
Initializing random number generator from current time

Prime numbers limit: 20000

Initializing worker threads...

Threads started!

CPU speed:
  events per second:   310.44

General statistics:
  total time:          10.0031s
  total number of events: 3106

Latency (ms):
  min:                 3.09
  avg:                 3.22
  max:                 5.72
  95th percentile:    3.36
  sum:                 9996.97

Threads fairness:
  events (avg/stddev): 3106.0000/0.00
  execution time (avg/stddev): 9.9970/0.00

root@2a9ac761ddcb:/app#
```

Imagen 12: Resultados sin limitar las CPUs

```
root@e8d381a937d7: /app
>> docker run -it --cpus=".5" waving969/sysbench-ubuntu bash
root@e8d381a937d7:/app# sysbench --test=cpu --cpu-max-prime=20000 run
WARNING: the --test option is deprecated. You can pass a script name or path on the command line without any options.
sysbench 1.0.18 (using system LuaJIT 2.1.0-beta3)

Running the test with following options:
Number of threads: 1
Initializing random number generator from current time

Prime numbers limit: 20000

Initializing worker threads...

Threads started!

CPU speed:
  events per second:   152.87

General statistics:
  total time:          10.0258s
  total number of events: 1533

Latency (ms):
  min:                 3.11
  avg:                 6.54
  max:                 122.23
  95th percentile:    52.89
  sum:                 10020.76

Threads fairness:
  events (avg/stddev): 1533.0000/0.00
  execution time (avg/stddev): 10.0208/0.00
root@e8d381a937d7:/app#
```

Imagen 13: Resultados limitando las CPU a 5

Como podemos observar los eventos por segundo se ha reducido a la mitad y también han aumentado la latencia, por lo que podemos ver cómo se han limitado las CPUs correctamente.